

Introduction aux modèles économétriques avec du Machine Learning

Correction Méthodes de Régularisation (Ridge, Lasso, Elastic Net, Post-Lasso)

Agossou M. YOMAKOU

2025-05-08



University
Mohammed VI
Polytechnic



FACULTÉ DE GOUVERNANCE,
SCIENCES ECONOMIQUES ET SOCIALES

Contents

1	Modèle Ridge	3
2	Modèle Lasso	4
3	Elastic Net	5
4	Post Lasso	5
5	Evaluation	6
6	Stabilité	8
7	Interpretation	9

Q1 : Standardisation cruciale pour comparer les pénalités

Réponse : Les méthodes sont sensibles à l'échelle des variables

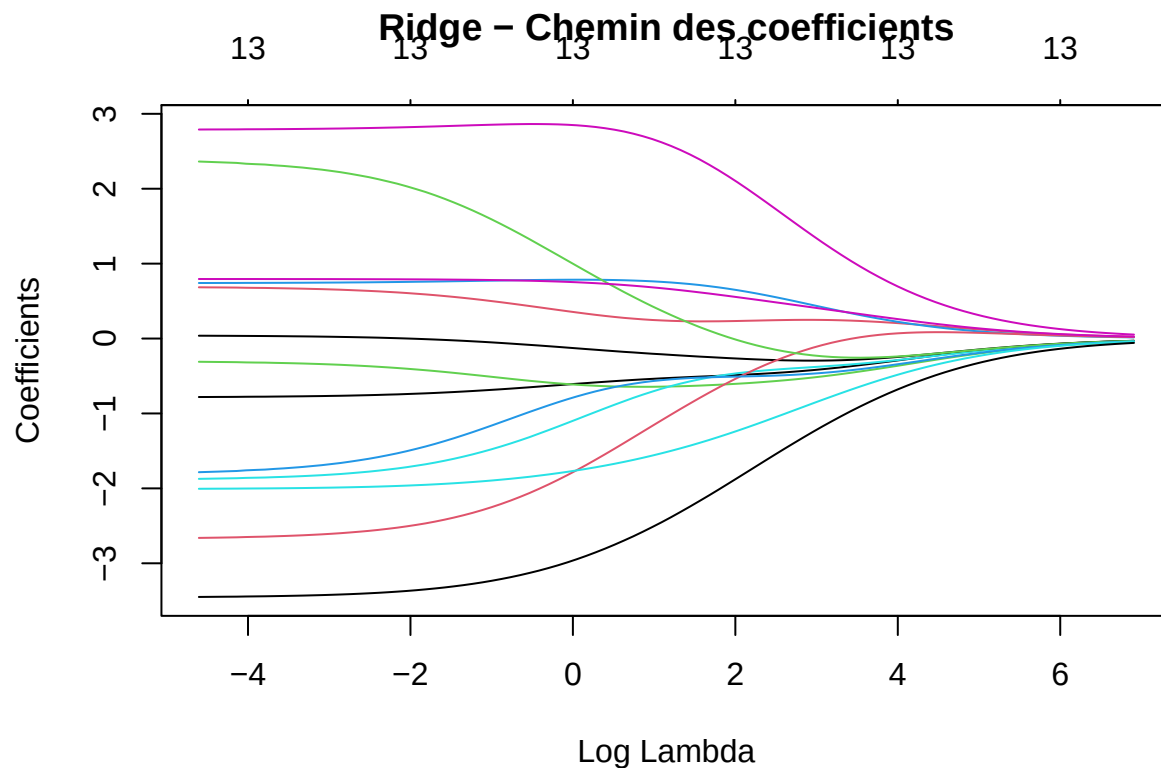
```
# Q2 : Standardisation
data(Boston, package = "MASS")
X <- scale(Boston[, -14]) # Standardisation des prédicteurs
y <- Boston$medv          # Variable réponse non standardisée

# Q3 : Découpage train/test
set.seed(123)
train_index <- createDataPartition(y, p = 0.7, list = FALSE)
X_train <- X[train_index, ]
y_train <- y[train_index]
X_test <- X[-train_index, ]
y_test <- y[-train_index]
```

1 Modèle Ridge

```
set.seed(123)
# Q4 : Modèle Ridge (alpha=0)
lambda_grid <- 10^seq(3, -2, length = 100)
ridge <- cv.glmnet(X_train, y_train, alpha = 0, lambda = lambda_grid)

# Q5 : Chemin des coefficients
plot(ridge$glmnet.fit, xvar = "lambda", main = "Ridge - Chemin des coefficients")
```

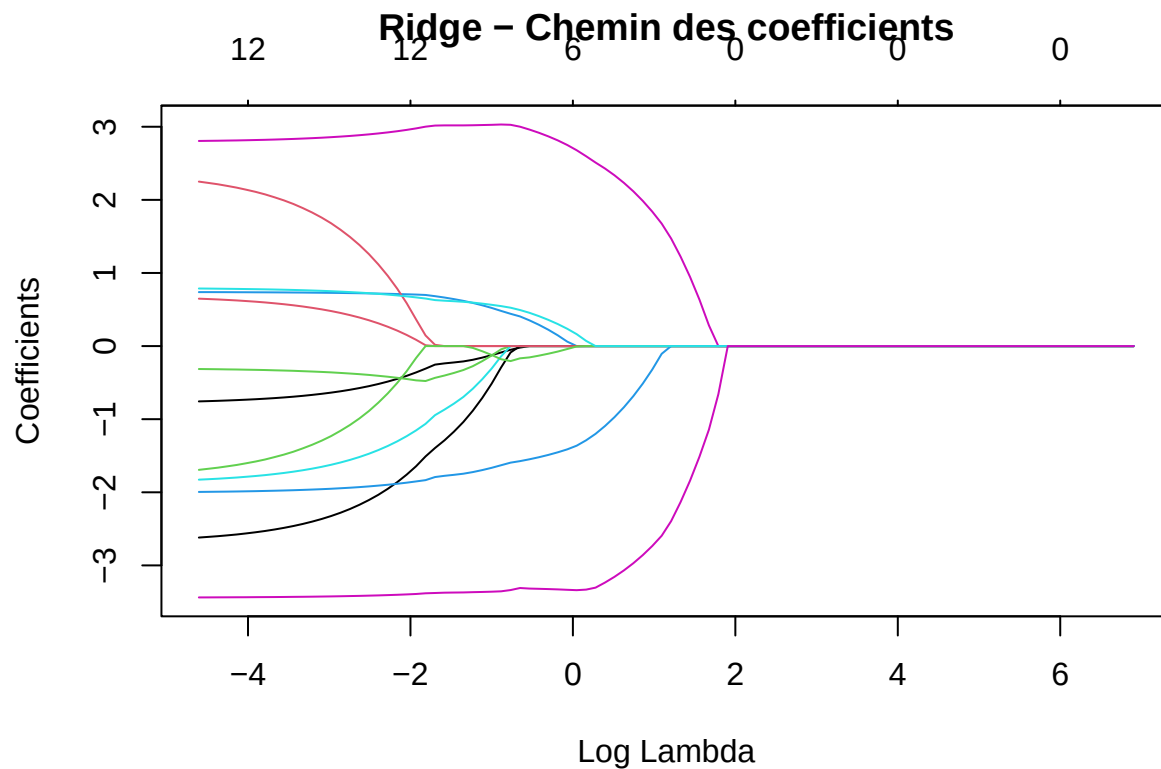


```
# Q6 : lambda optimal
best_lambda_ridge = ridge$lambda.min
cat("lambda optimal (Ridge):", ridge$lambda.min)
```

```
## lambda optimal (Ridge): 0.231013
```

2 Modèle Lasso

```
# Q7 : Modèle Lasso (alpha=1)
lasso <- cv.glmnet(X_train, y_train, alpha = 1, lambda = lambda_grid)
best_lambda_lasso <- lasso$lambda.min
plot(lasso$glmnet.fit, xvar = "lambda", main = "Ridge - Chemin des coefficients")
```



```
# Nombre de coefs nuls pour lambda=0.1
coef_at_0.1 <- predict(lasso, type = "coefficients", s = 0.1)
sum(coef_at_0.1[-1] == 0) # Exclut l'intercept
```

```
## [1] 1
```

```
# Q8 : Comparaison avec VIF
model_lm <- lm(medv ~ ., data = Boston[train_index, ])
car::vif(model_lm) # Forte colinéarité si VIF > 5-10
```

```
##      crim      zn      indus      chas      nox      rm      age      dis
```

```
## 1.803945 2.439767 3.997561 1.088492 4.496675 1.897191 3.268411 4.218721
##      rad      tax ptratio   black   lstat
## 7.119581 8.293614 1.772110 1.382755 3.239807
```

Q9 : Avantage du Lasso

Réponse : Le Lasso élimine les variables redondantes (coefs=0), contrairement à Ridge qui ne fait que les réduire

3 Elastic Net

```
# Q10 : Optimisation de alpha
set.seed(123)
enet <- train(
  x = X_train,
  y = y_train,
  method = "glmnet",
  tuneGrid = expand.grid(
    alpha = seq(0, 1, length = 10), # Grille pour alpha
    lambda = lambda_grid
  ),
  trControl = trainControl("cv", number = 10)
)

best_alpha <- enet$bestTune$alpha
best_lambda_enet <- enet$bestTune$lambda

# La meilleure combinaison de alpha et de lambda pour l'elastic net
enet$bestTune
```

```
##      alpha   lambda
## 125 0.1111111 0.1629751
```

```
# Q11 : Nombre de variables sélectionnées
coef_enet <- coef(enet$finalModel, s = best_lambda_enet)
sum(coef_enet[-1] != 0) # Variables non nulles
```

```
## [1] 12
```

Pour avoir tout les résultats de combinaison de α et de λ pour l'Elastic Net, la commande est `enet$results`. Elle affichera

4 Post Lasso

```
# Q12 : Variables sélectionnées par Lasso
selected_vars <- which(coef(lasso, s = "lambda.min")[-1] != 0)
colnames(X_train[, selected_vars]) # Affiche les noms
```

```
## [1] "crim"      "zn"        "indus"     "chas"      "nox"       "rm"        "dis"
## [8] "rad"       "tax"       "ptratio"   "black"     "lstat"
```

```
# Q13 : Réestimation par OLS
X_sel <- X_train[, selected_vars, drop = FALSE]
post_lasso <- lm(y_train ~ ., data = as.data.frame(X_sel))
summary(post_lasso) # Comparez les coefs avec ceux du Lasso
```

```
##
## Call:
## lm(formula = y_train ~ ., data = as.data.frame(X_sel))
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -15.5083  -2.7587  -0.5059   1.9547  24.5645
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  22.5185     0.2447   92.007 < 2e-16 ***
## crim        -0.7878     0.3325   -2.369 0.018373 *
## zn           0.6920     0.3774    1.834 0.067585 .
## indus       -0.2966     0.4857   -0.611 0.541748
## chas         0.7403     0.2530    2.926 0.003657 **
## nox         -1.8792     0.4964   -3.786 0.000181 ***
## rm           2.7930     0.3119    8.954 < 2e-16 ***
## dis         -2.6914     0.4634   -5.808 1.44e-08 ***
## rad          2.4047     0.6435    3.737 0.000218 ***
## tax         -1.8224     0.6988   -2.608 0.009507 **
## ptratio     -2.0070     0.3263   -6.150 2.16e-09 ***
## black        0.7968     0.2720    2.930 0.003621 **
## lstat       -3.4401     0.4067   -8.459 7.90e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4.605 on 343 degrees of freedom
## Multiple R-squared:  0.7481, Adjusted R-squared:  0.7393
## F-statistic: 84.91 on 12 and 343 DF,  p-value: < 2.2e-16
```

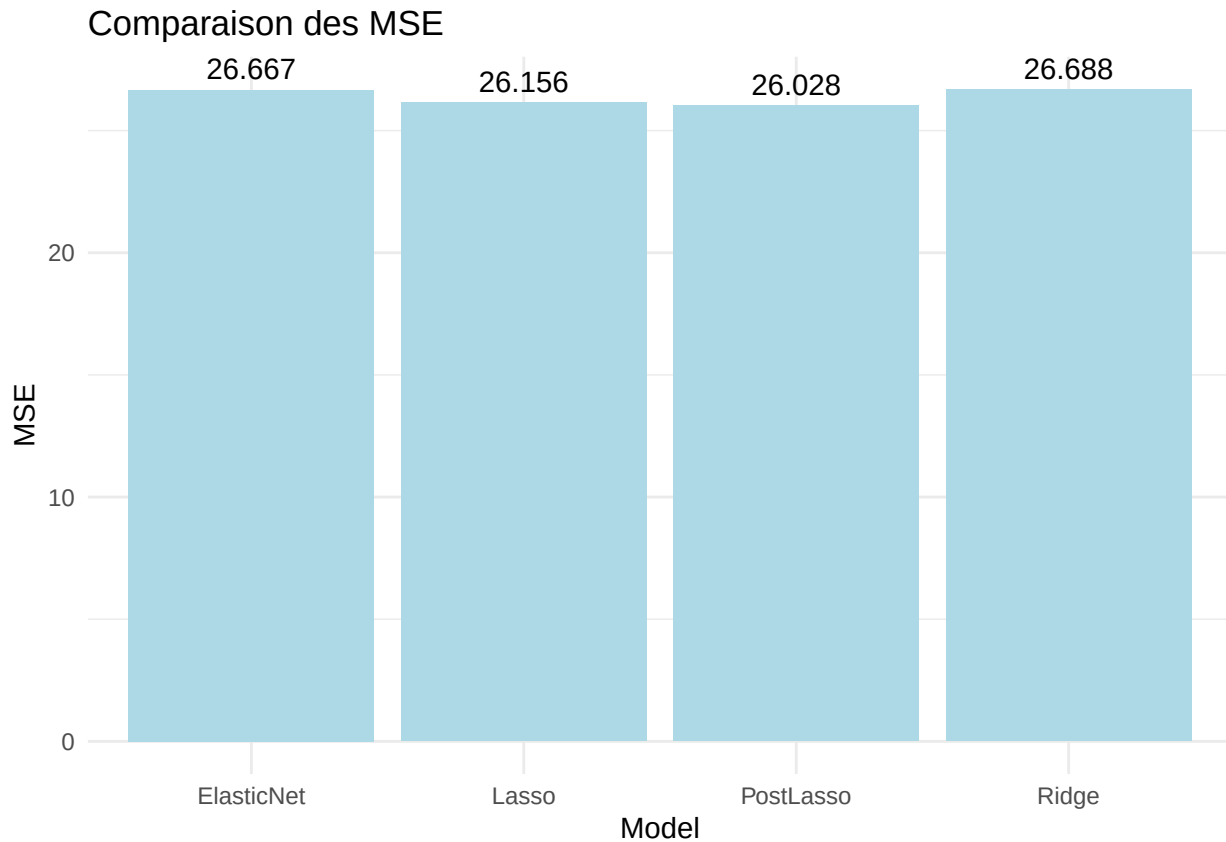
5 Evaluation

```
# Q14 : Calcul des MSE
predictions <- list(
  Ridge = predict(ridge, s = best_lambda_ridge, newx = X_test),
  Lasso = predict(lasso, s = best_lambda_lasso, newx = X_test),
  ElasticNet = predict(enet, newdata = X_test),
  PostLasso = predict(post_lasso, newdata = data.frame(X_test))
)

mse_results <- sapply(predictions, function(pred) mean((y_test - pred)^2))

# Q15 : Visualisation
```

```
ggplot(data.frame(Model = names(mse_results), MSE = mse_results),
       aes(x = Model, y = MSE)) +
  geom_col(fill = "lightblue") +
  geom_text(aes(label = round(MSE, 3)), vjust = -0.5) +
  labs(title = "Comparaison des MSE") +
  theme_minimal()
```



```
post_coef <- coef(post_lasso)

# 2. Nettoyer les noms en supprimant le préfixe
clean_names <- gsub("X_train\\[, selected_vars\\]", "", names(post_coef))
post_lasso_coef <- rep(NA, ncol(X))
for(i in seq_along(clean_names[-1])) { # Exclure l'intercept
  var_name <- clean_names[-1][i]
  if(var_name %in% colnames(X)) {
    post_lasso_coef[which(colnames(X) == var_name)] <- post_coef[-1][i]
  }
}

# Q16 : Comparaison des coefficients
coef_comparison <- data.frame(
  Variable = colnames(X),
  Ridge = as.vector(coef(ridge, s = "lambda.min")[-1]),
  Lasso = as.vector(coef(lasso, s = "lambda.min")[-1]),
  ElasticNet = as.vector(coef(enet$finalModel, s = best_lambda_enet)[-1]),
  PostLasso = post_lasso_coef
```

```
)
coef_comparison
```

##	Variable	Ridge	Lasso	ElasticNet	PostLasso
## 1	crim	-0.71394002	-0.7528386	-0.6900378	-0.7877729
## 2	zn	0.55566421	0.6432967	0.5322256	0.6919598
## 3	indus	-0.45803897	-0.3153325	-0.4255087	-0.2966456
## 4	chas	0.76499792	0.7380557	0.7514908	0.7402652
## 5	nox	-1.60485582	-1.8208187	-1.6223520	-1.8791642
## 6	rm	2.83928642	2.8073462	2.8417374	2.7929996
## 7	age	-0.02409574	0.0000000	0.0000000	NA
## 8	dis	-2.39008791	-2.6101204	-2.3700475	-2.6913974
## 9	rad	1.81677536	2.2319427	1.7867071	2.4047488
## 10	tax	-1.33085405	-1.6783570	-1.3077088	-1.8223819
## 11	ptratio	-1.93186164	-1.9936291	-1.9412828	-2.0069587
## 12	black	0.78659238	0.7862482	0.7735432	0.7968195
## 13	lstat	-3.30812600	-3.4374501	-3.3549231	-3.4400953

La comparaison des coefficients estimés par les méthodes Ridge, Lasso, ElasticNet et PostLasso met en évidence plusieurs points intéressants. Certaines variables, comme `rm` (nombre de pièces), `dis` (distance aux centres d'emploi), et `lstat` (pourcentage de population à faible statut socio-économique), ressortent systématiquement comme très influentes, avec des effets forts et stables entre les modèles. Cela confirme leur rôle déterminant dans l'explication du prix des logements. D'autres variables comme `indus` (proportion de terrains non résidentiels) sont exclues par Lasso et PostLasso, suggérant une faible pertinence prédictive ou une colinéarité avec d'autres variables. Ridge, contrairement à Lasso, conserve toutes les variables mais tend à réduire l'amplitude des coefficients, tandis que Lasso fait une sélection automatique en mettant certains à zéro. ElasticNet offre un compromis entre ces deux approches, et PostLasso affine les coefficients estimés après la sélection faite par Lasso.

6 Stabilité

```
# Q17
n_boot <- 100
selected_counts <- matrix(0, nrow = ncol(X), dimnames = list(colnames(X), NULL))

set.seed(123)
for (i in 1:n_boot) {
  idx <- sample(nrow(X_train), replace = TRUE)
  boot_fit <- cv.glmnet(X_train[idx, ], y_train[idx], alpha = 1)
  selected <- which(coef(boot_fit, s = "lambda.min")[-1] != 0)
  selected_counts[selected] <- selected_counts[selected] + 1
}

# Fréquence de sélection
data.frame(Variable = rownames(selected_counts),
           Freq = selected_counts/n_boot) %>%
  arrange(desc(Freq))
```

```
##          Variable Freq
```

```
## rm          rm 1.00
## dis         dis 1.00
## ptratio    ptratio 1.00
## black      black 1.00
## lstat      lstat 1.00
## chas       chas 0.99
## nox        nox 0.99
## rad        rad 0.98
## crim       crim 0.95
## tax        tax 0.95
## zn         zn 0.93
## indus      indus 0.87
## age        age 0.80
```

7 Interpretation

Les variables les plus fiables pour la modélisation semblent être celles avec une fréquence de 1.00. Celles avec une fréquence plus faible méritent une analyse complémentaire.

Q18 :

Réponse:

- Post-Lasso recalcule les coefs sans pénalité, réduisant le biais
- Ridge préférable quand toutes les variables sont pertinentes